Module 5

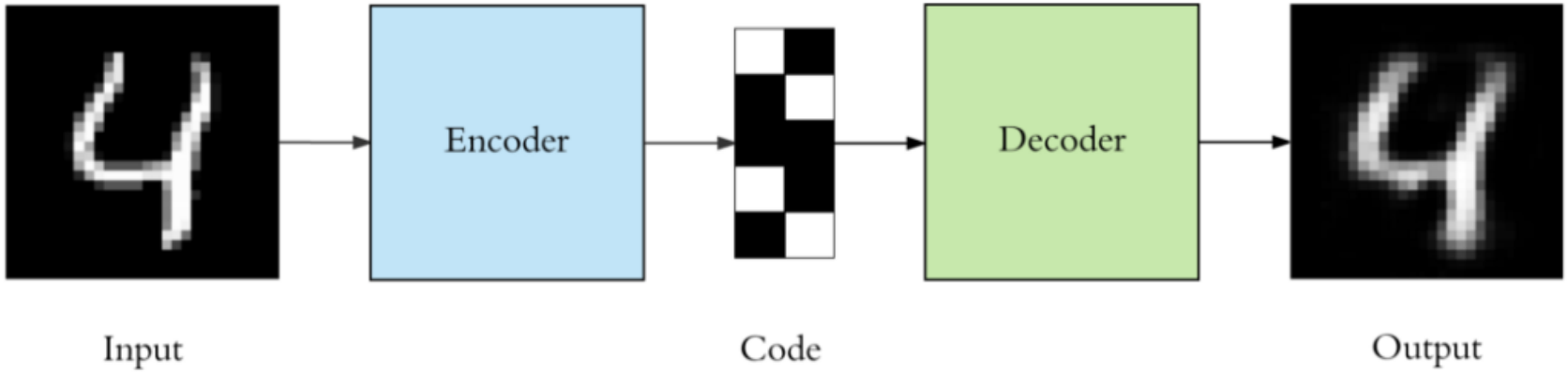
Autoencoders

- Autoencoders are a specific type of feedforward neural networks where the input is the same as the output.
- They compress the input into a **lower-dimensional *code*** and then **reconstruct the output from this representation**.
- The code is a compact “summary” or “compression” of the input, also called the ***latent-space representation***


Original Image
Autoencoder
PCA

Autoencoders

- An autoencoder consists of encoder ,code and decoder.
- The encoder compresses the input and produces the code. It works as a compression unit that compress the input data.
- The decoder decompresses the compressed input by reconstructing it.



3 Components of an Autoencoder

1. Encoder

- Part of the network that compress the input into a compressed form known as **latent space representation**.

2. Code - This is the part of the network which represents the compressed input that is fed to the decoder. The layer between the encoder and decoder, ie. the code is also known as **Bottleneck**. It decide which aspects of observed data are relevant information and what aspects can be discarded.

3.Decoder – It aims to reconstruct input from the latent space representation.

- To build an autoencoder , 3 things are needed:
    1. An encoding method
    2. A decoding method,
    3. A loss function to compare the output with the target.

Properties of Autoencoders

Autoencoders are mainly a **dimensionality reduction (or compression) algorithm** with following important properties:

1. **Data-specific:**

Autoencoders are able to meaningfully compress only the type of data similar to what they have been trained on.

Autoencoders are different than a standard data compression algorithms, since they learn features that are specific to a given training data. So an autoencoder trained on handwritten digits can not be used to compress landscape photos.

2. Lossy:

The output of the autoencoder will not be exactly the same as the input, it will be a close but degraded representation.

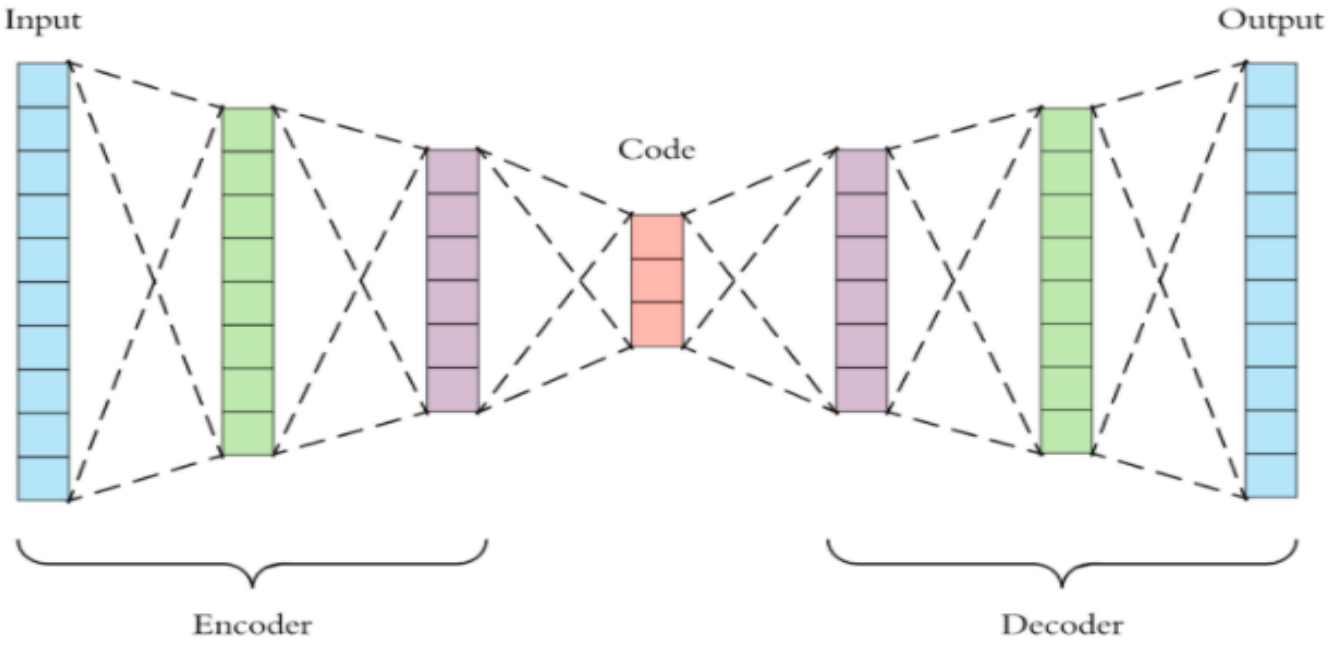
3. Unsupervised:

To train an autoencoder, only thing to be done is provide it with raw input data.

Autoencoders are considered an *unsupervised* learning technique since they don't need explicit labels to train on. But to be more precise they are *self-supervised* because they generate their own labels from the training data.

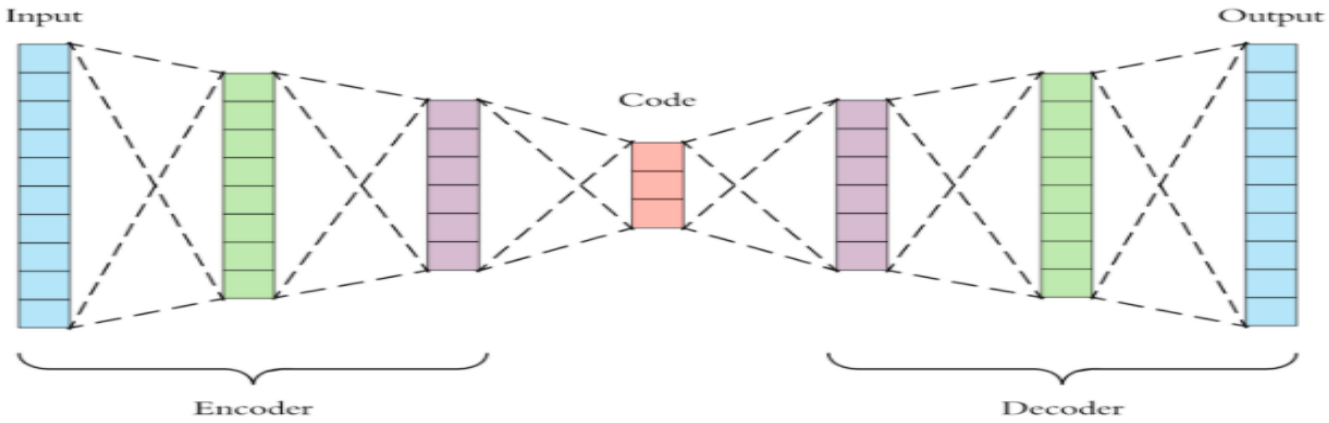
Architecture of Autoencoders(Training Autoencoders)

- Both the encoder and decoder are fully-connected feedforward neural networks,
- Code is a single layer of an ANN with the dimensionality of our choice. Ie. the number of nodes in the code layer (code size) is a *hyperparameter* that we set before training the autoencoder



- First the input passes through the encoder, which is a fully-connected ANN, to produce the code.
- The decoder then produces the output only using the code. Decoder has the similar ANN structure (fully-connected ANN) that of encoder.
- The goal of autoencoders is to get an output identical with the input.
- The dimensionality of the input and output needs to be the same.

Note: ***The decoder architecture is the mirror image of the encoder. This is not a requirement. The only requirement is the dimensionality of the input and output needs to be the same.***



There are 4 hyperparameters that need to set before training an autoencoder:

1. Code size:

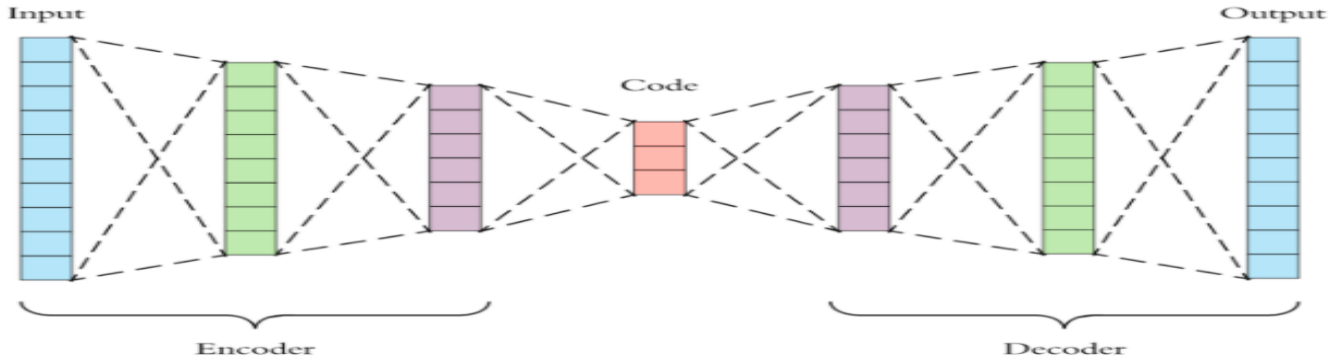
   Code size is the number of nodes in the middle layer.

   Smaller size results in more compression.

2. Number of layers in encoder and decoder :

   The autoencoder can be as deep as we like.

   Eg: In the figure above we have 2 layers in both the encoder and decoder, without considering the input and output.



3. Number of nodes per layer:

- The number of nodes per layer decreases with each subsequent layer of the encoder, and increases back in the decoder.

-The decoder is symmetric to the encoder in terms of layer structure.

The Autoencoder architecture is called a stacked autoencoder since the layers are stacked one after another.

Note

- Autoencoder is an **unsupervised ML algorithm** similar to Principal Component Analysis(PCA). Autoencoders are used to reduce the size of the inputs into a smaller representation. If anyone needs the original data, they can reconstruct it from the compressed data.
- It minimizes the same objective function as PCA
- It is **a neural network** and Target output of this neural network is its input.
- To train an Autoencoder, backpropagation algorithm is used which try to **minimize reconstruction error which is represented as**

$E(X, \hat{X})$ where X is original image, $\hat{X}$ is reconstructed Image

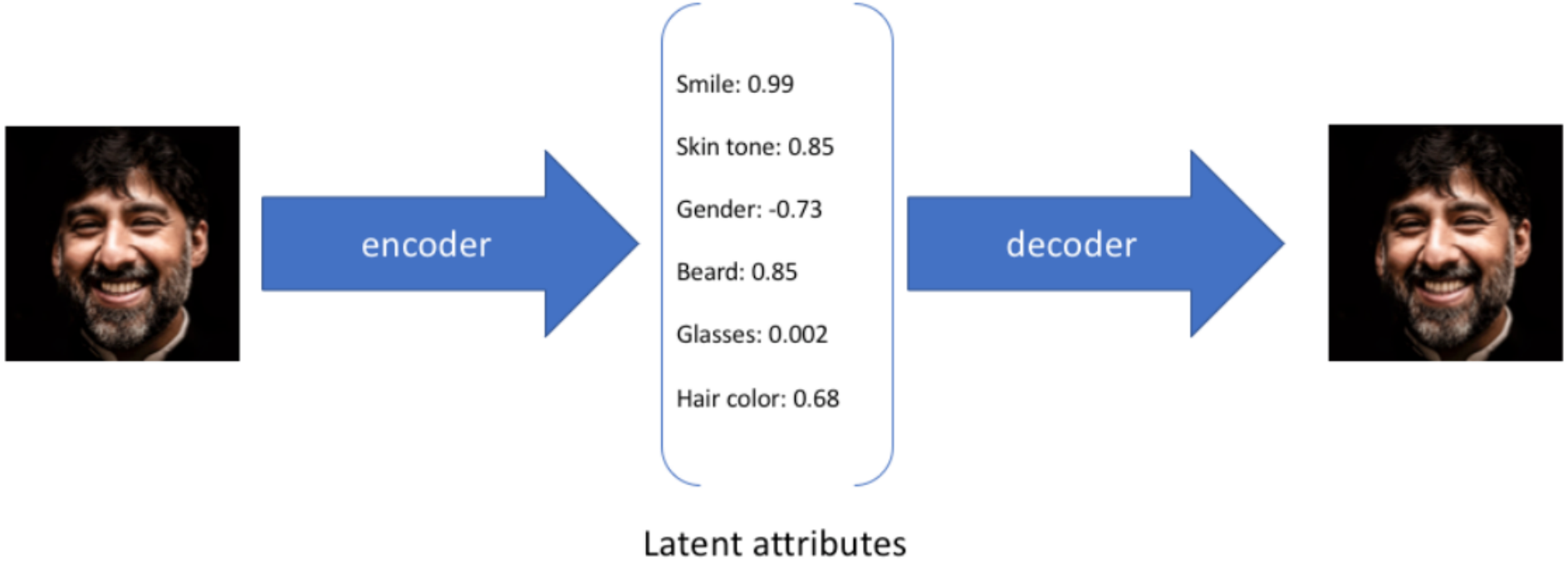
Variational Autoencoders (VAEs)

- Variational Autoencoders (VAEs) are the most effective and useful process for **Generative Models.**
- Generative models are used for generating new synthetic or artificial data from the real data. For example, new music composition from currently composed music.

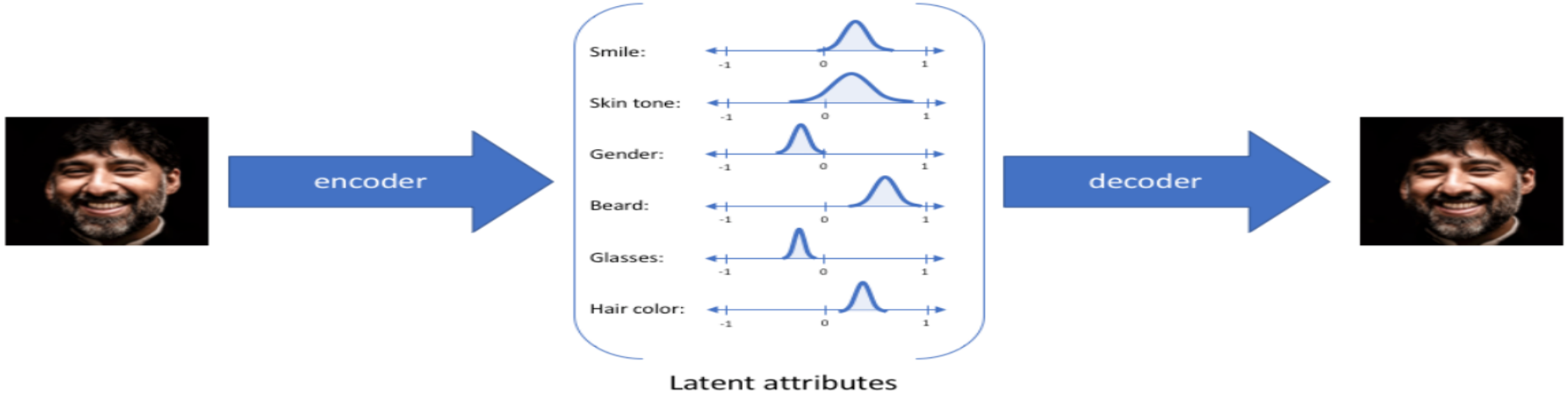
- In Autoencoders, input data is converted into an *encoding vector* where each dimension represents some learned attribute about the data.
- The encoder outputs a *single value* for each attribute.
- The decoder network takes these values and attempts to recreate the original input.

- Suppose an autoencoder model is trained on a large dataset of faces with an encoding dimension of 6. The autoencoder will learn attributes of faces such as skin color, whether or not the person is wearing glasses, etc. in an attempt to describe an observation in some compressed representation.

The input image is described in terms of its latent attributes and a single value is used to describe each attribute.



- In a Variational Auto Encoder(VAE), each latent attribute in the input image is described as a range of possible values.









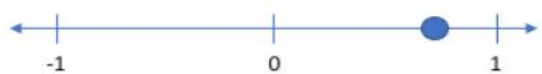


Smile (discrete value)



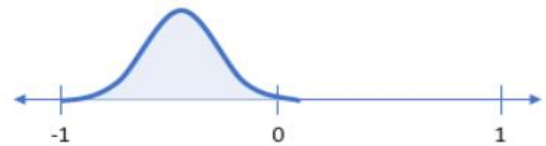


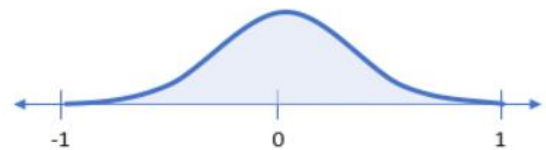


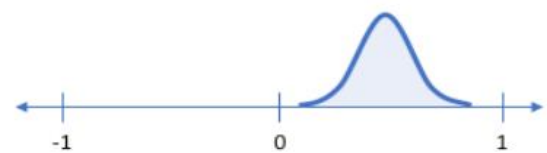


vs.

Smile (probability distribution)









- For variational autoencoders, the encoder model is sometimes referred to as the **recognition model** whereas the decoder model is sometimes referred to as the **generative model.**

Applications of Autoencoders

- The primary applications of an autoencoder is for
- **A**nomaly detection or image denoising.
- **Other applications of Autoencoders are**
- Dimensionality Reduction.
- Image Compression.
- Feature Extraction.
- Image generation.
- Sequence to sequence prediction.
- Recommendation system.

2)Image Compression

- Usually, Autoencoders are really not good for data compression.
  For Image Compression, it is pretty difficult for an autoencoder to do better than basic algorithms, like JPEG and by being only specific for a particular type of images.

- Data-specific property of autoencoders makes it impractical for compression of real-world data. One can only use them for data on which they were trained, and therefore, generalisation requires a lot of data.

3) Image Denoising :

To obtain proper information about the content of image, **Image Denoising** is required. An autoencoder can be used to remove most of the noise of the image.

A denoising autoencoder is trained to reconstruct the original input from the noisy version.

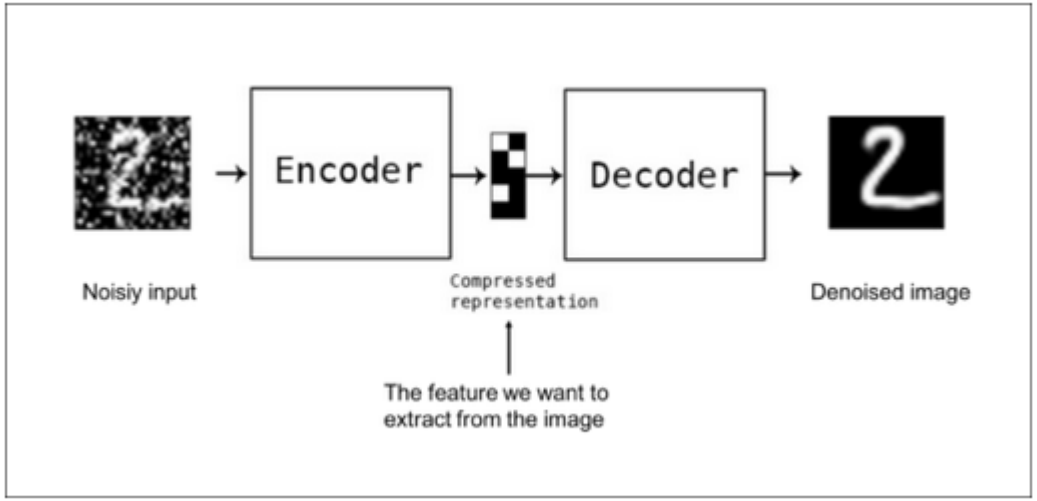
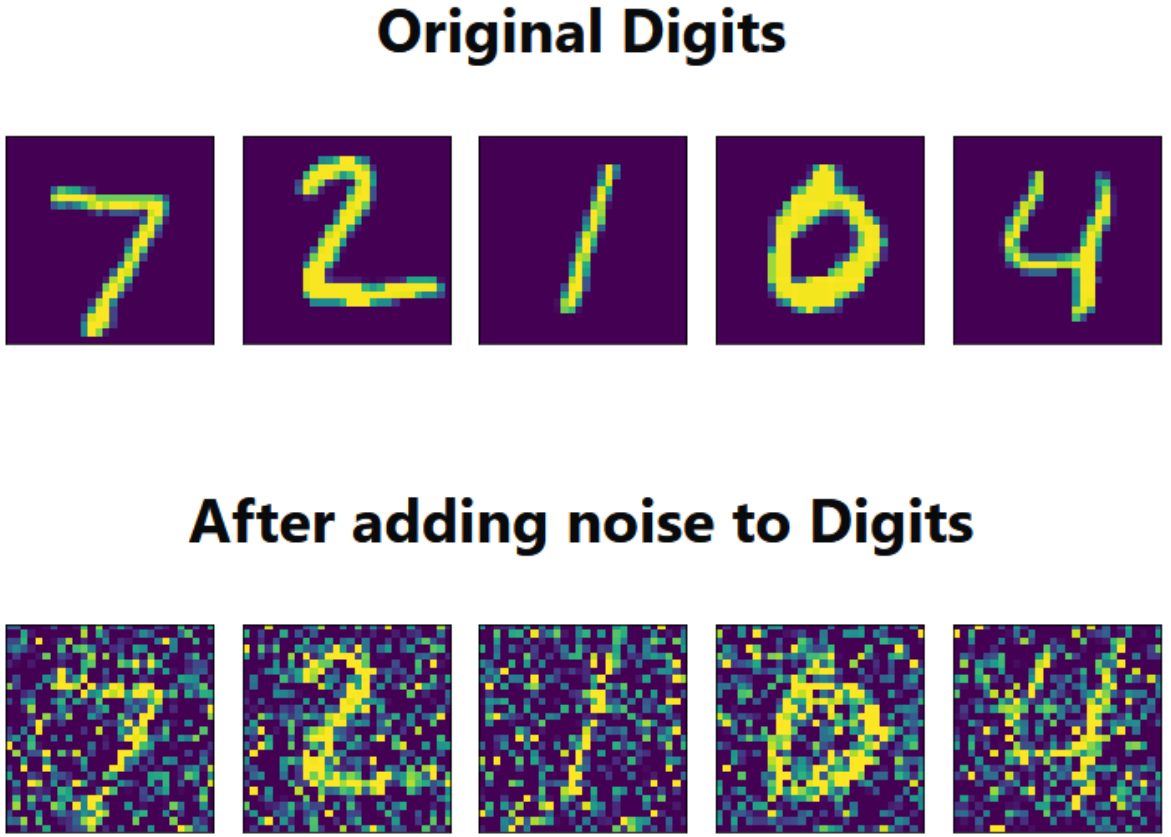
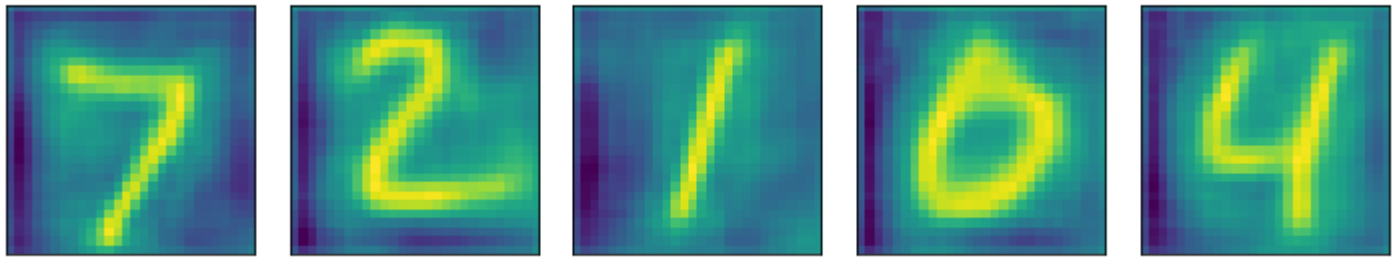


Image denoising





3) Image Denoising :

To obtain proper information about the content of image, **Image Denoising** is required. An autoencoder can be used to remove most of the noise of the image.

4) Feature Extraction

Encoding part of Autoencoders helps to learn important hidden features present in the input data, in the process to reduce the reconstruction error. During encoding, a new set of combination of original features is generated.

5) Image Generation

- There is a type of Autoencoder, named Variational Autoencoder(VAE), this type of autoencoders are **Generative Model**, used to generate images.
- The idea is that given input images like images of face or scenery, the system will generate similar images. The use is to:
- generate new characters of animation
- generate fake human images



6) Sequence to Sequence Prediction

- The Encoder-Decoder Model like LSTMs-based autoencoders, can be used to
- Address Machine Translation problems.
- Predict the next frame of a video
- Generate fake videos

7.Recommendation Systems

- Autoencoders can be used to understand user preferences to recommend movies, books or other items.
- In the case of YouTube Videos,
- the input data is the clustering of similar users based on interests
- interests of users are denoted by

✓ videos watched,

✓ watch time for each

✓ interactions (like commenting) with the video

- above data is captured and Encoder part will capture the interests of the user
- Decoder part will try to project the interests on two parts:
  - existing unseen content
  - new content from content creators

8) Watermark Removal

- It is also used for removing watermarks from images or to remove any object while filming a video or a movie.



9)Image Coloring

- Autoencoders are used for converting any black and white picture into a colored image. Depending on what is in the picture, it is possible to tell what the color should be.



GAN(Generative Adversarial Networks)

- Generative Adversarial Networks(GANs)
- -an approach to generative modeling using deep learning methods, such as convolutional neural networks.
- Generative modeling is an unsupervised learning task in machine learning that involves automatically discovering and learning the regularities or patterns in input data in such a way that the model can be used to generate or output new examples that probably could have been drawn from the original dataset.

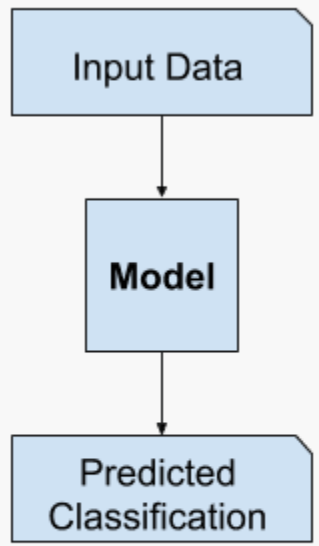
- Machine learning models can be classified into two types –
- Discriminative and
- Generative models.
- A discriminative model makes predictions on the unseen data based on conditional probability and can be used either for classification or regression problem statements.
- On the contrary, a generative model focuses on the distribution of a dataset to return a probability for a given example.

Discriminative vs. Generative Modeling

- **Generative** models can generate new data instances.
- **Discriminative** models discriminate between different kinds of data instances.
- A generative model could generate new photos of animals that look like real animals, while a discriminative model could tell a dog from a cat. GANs are just one kind of generative model.

- **Discriminative Models**

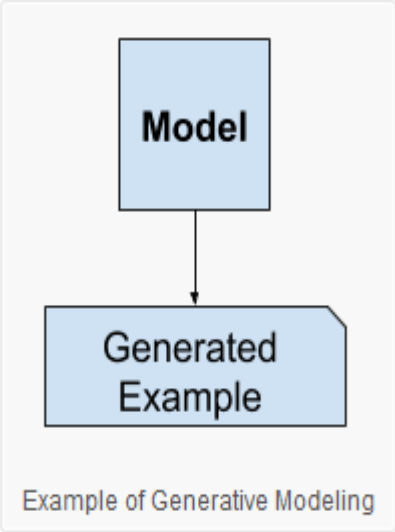
- In supervised learning, a machine learning model is developed to predict a class label for a given example of input variables. This predictive modeling task is called classification.
- Classification is also traditionally referred to as discriminative modeling.
- The training data is used to find a discriminant function f(x) that maps each x directly onto a class label, thereby combining the inference and decision stages into a single learning problem.
- A model must discriminate examples of input variables across classes; it must choose or make a decision as to what class a given example belongs.



Example of Discriminative Modeling

Generative models

- Generative models are unsupervised models that summarize the distribution of input variables and it may be able to generate new examples that are not indistinguishable from real examples from the problem domain.



Example of Generative Modeling

Examples of Generative Models

- Naïve Bayes
- Bayesian networks
- Hidden Markov Models (HMMs)
- Latent Dirichlet Allocation (LDA)
- Variational Autoencoder, or VAE
- Generative Adversarial Networks (GANs)

Examples of Discriminative Models

- Logistic regression
- Scalar Vector Machine (SVMs)
- Traditional neural networks
- Nearest neighbor
- Conditional Random Fields (CRFs)
- Decision Trees and Random Forest

Comparison between Discriminative and Generative Models

- To compare between Discriminative and Generative Models following criterias are used
- Performance
- Missing Data
- Accuracy Score
- Applications
- **Based on Performance**
- Generative models need fewer data to train compared with discriminative models since generative models are more biased as they make stronger assumptions.
- **Based on Missing Data**
- In general, if missing data is in the dataset, then Generative models can work with these missing data, while on the contrary discriminative models can not. For discriminative models, we usually require all the features X to be observed.

- **Based on Accuracy Score**
- Generative models are less accurate than discriminative models.
- **Based on Applications**
- Generative models are useful for unsupervised learning tasks while discriminative models are useful for supervised learning tasks.
- A generative model focuses on explaining how the data was generated, while a discriminative model focuses on predicting the labels of the data.
- Discriminative models can solve classification problems while Generative models have more applications besides classification.

GAN Structure

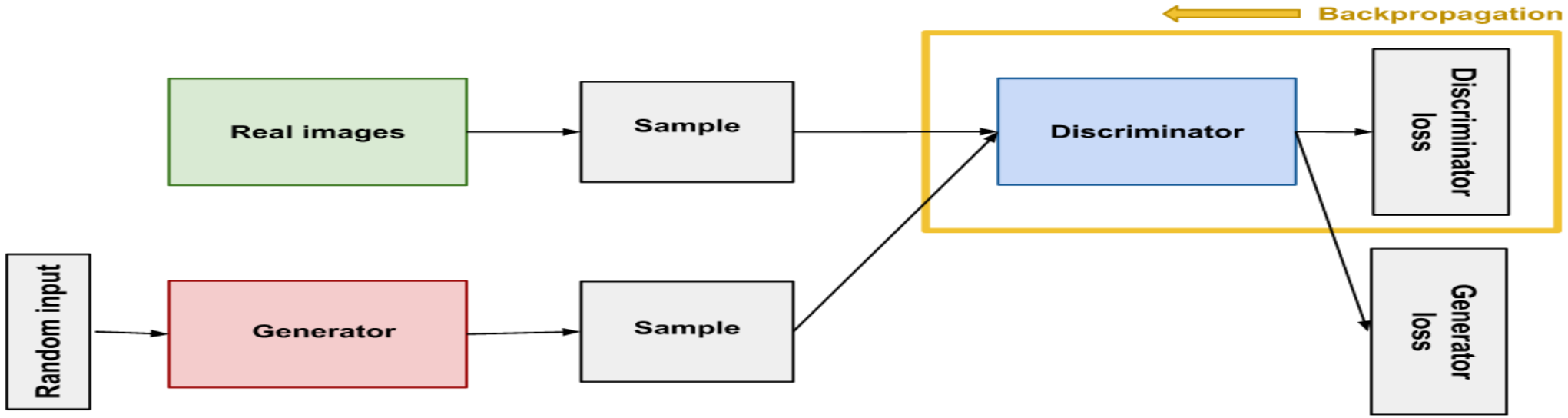
- A Generative adversarial network (GAN) has two parts:

✓Generator

✓Discriminator

GAN discriminator

- The discriminator in a GAN is simply a classifier. It tries to distinguish real data from the data created by the generator. It could use any network architecture appropriate to the type of data it's classifying.



Backpropagation in discriminator training.

In Figure, the two "Sample" boxes represent these two data sources feeding into the discriminator.

Discriminator Training Data

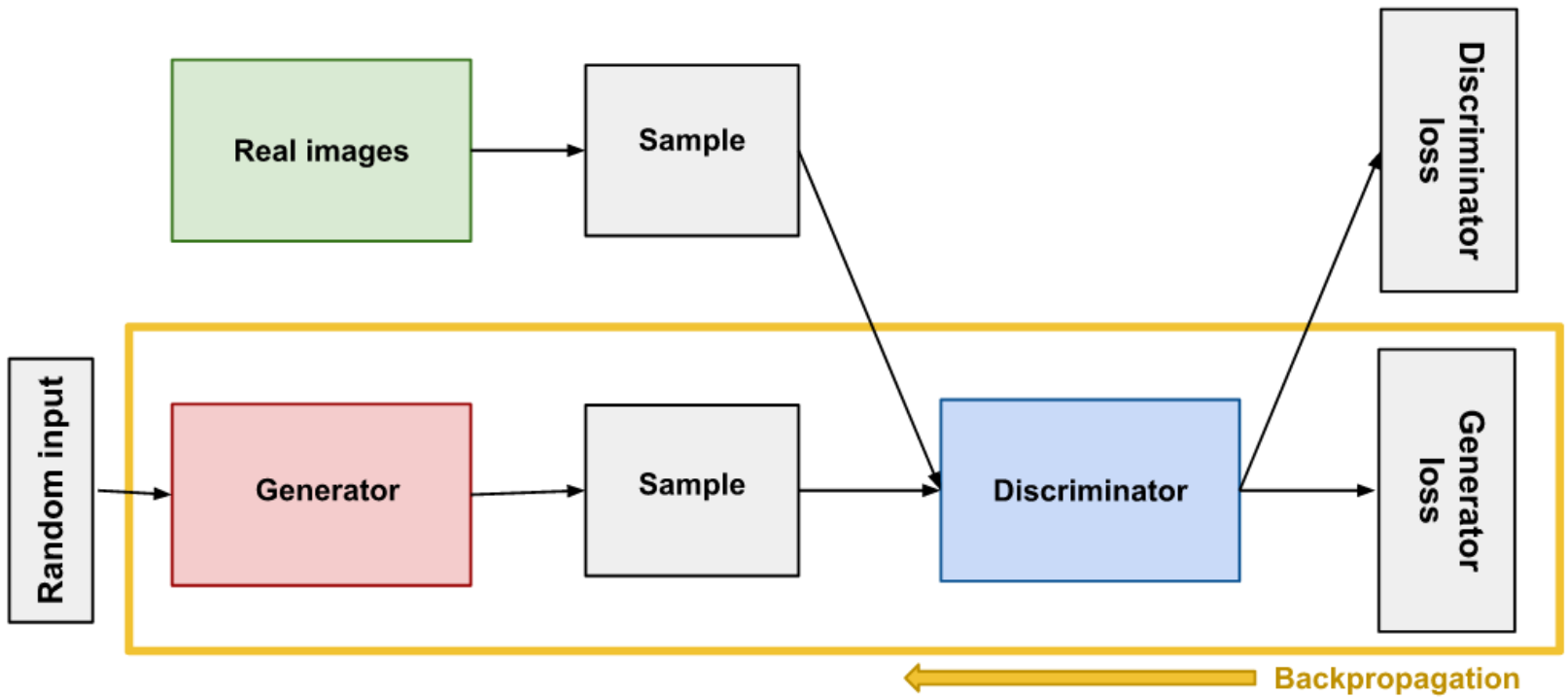
- The discriminator's training data comes from two sources:
- **Real data** instances, such as real pictures of people. The discriminator uses these instances as positive examples during training.
- **Fake data** instances created by the generator. The discriminator uses these instances as negative examples during training.
- During discriminator training the generator does not train. Its weights remain constant while it produces examples for the discriminator to train on.

- **Training the Discriminator**
- The discriminator is connected to two loss functions. During discriminator training, the discriminator ignores the generator loss and uses only the discriminator loss. During discriminator training:

1. The discriminator classifies both real data and fake data from the generator.
2. The discriminator loss penalizes the discriminator for misclassifying a real instance as fake or a fake instance as real.
3. The discriminator updates its weights through backpropagation from the discriminator loss through the discriminator network.

GAN Generator

- The generator part of a GAN learns to create fake data by incorporating feedback from the discriminator. It learns to make the discriminator classify its output as real.
- Generator training requires tighter integration between the generator and the discriminator than discriminator training requires. The portion of the GAN that trains the generator includes:
- random input
- generator network, which transforms the random input into a data instance
- discriminator network, which classifies the generated data
- discriminator output
- generator loss, which penalizes the generator for failing to fool the discriminator



Backpropagation in generator training.

- **Random Input**
- Neural networks need some form of input. A GAN takes random noise as its input.
- The generator then transforms this noise into a meaningful output. By introducing noise, we can get the GAN to produce a wide variety of data, sampling from different places in the target distribution.
- Usually the noise has smaller dimension than the dimensionality of the output space.

- The **generator** learns to generate plausible data. The generated instances become negative training examples for the discriminator.
- The **discriminator** learns to distinguish between the generator's fake data from real data. The discriminator penalizes the generator for producing implausible results.
- When training begins, the generator produces obviously fake data, and the discriminator quickly learns to tell that it's fake:

GAN Training

- Because a GAN contains two separately trained networks, its training algorithm must address two complications:
- GANs must juggle two different kinds of training (generator and discriminator).
- GAN convergence is hard to identify.

Alternating Training

- The generator and the discriminator have different training processes.
- GAN training proceeds in alternating periods:

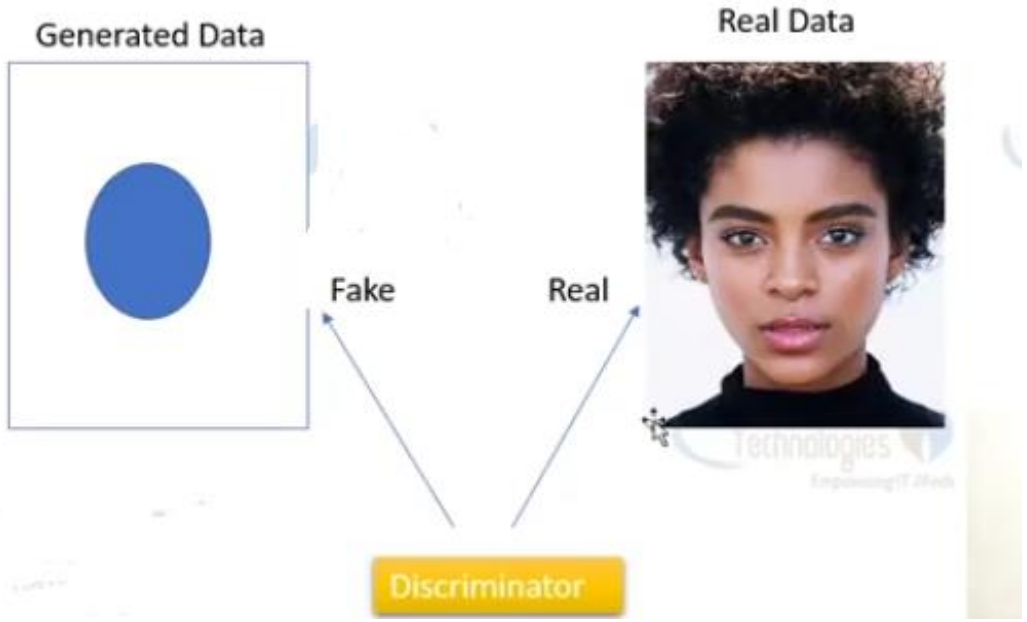
1.The discriminator trains for one or more epochs.

2.The generator trains for one or more epochs.

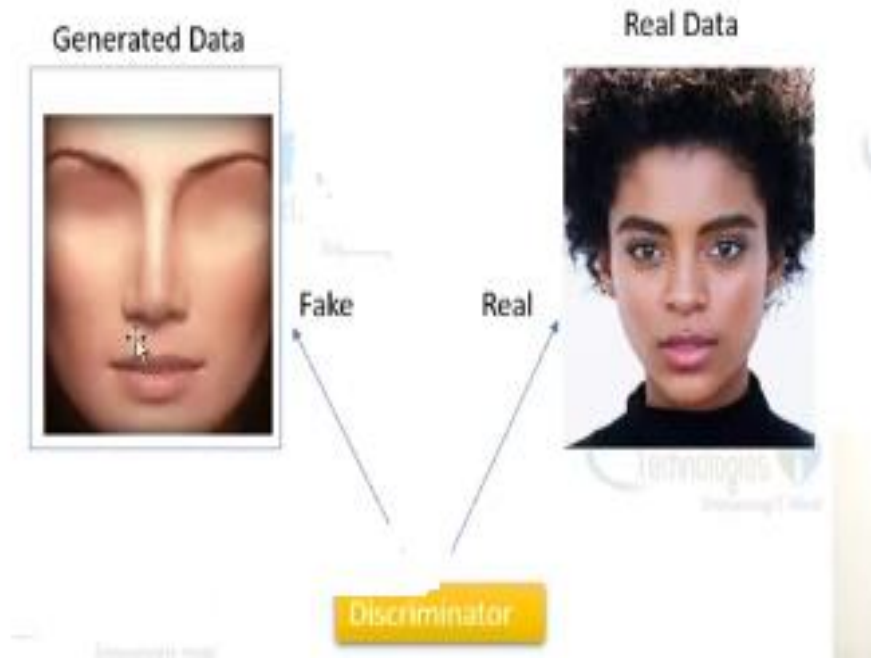
3.Repeat steps 1 and 2 to continue to train the generator and discriminator networks.

- The generator is kept constant during the discriminator training phase. As discriminator training tries to figure out how to distinguish real data from fake, it has to learn how to recognize the generator’s faults. That's a different problem for a thoroughly trained generator than it is for an untrained generator that produces random output.
- Similarly, the discriminator is kept constant during the generator training phase.
- It is this back and forth process that allows GANs to tackle otherwise intractable generative problems.

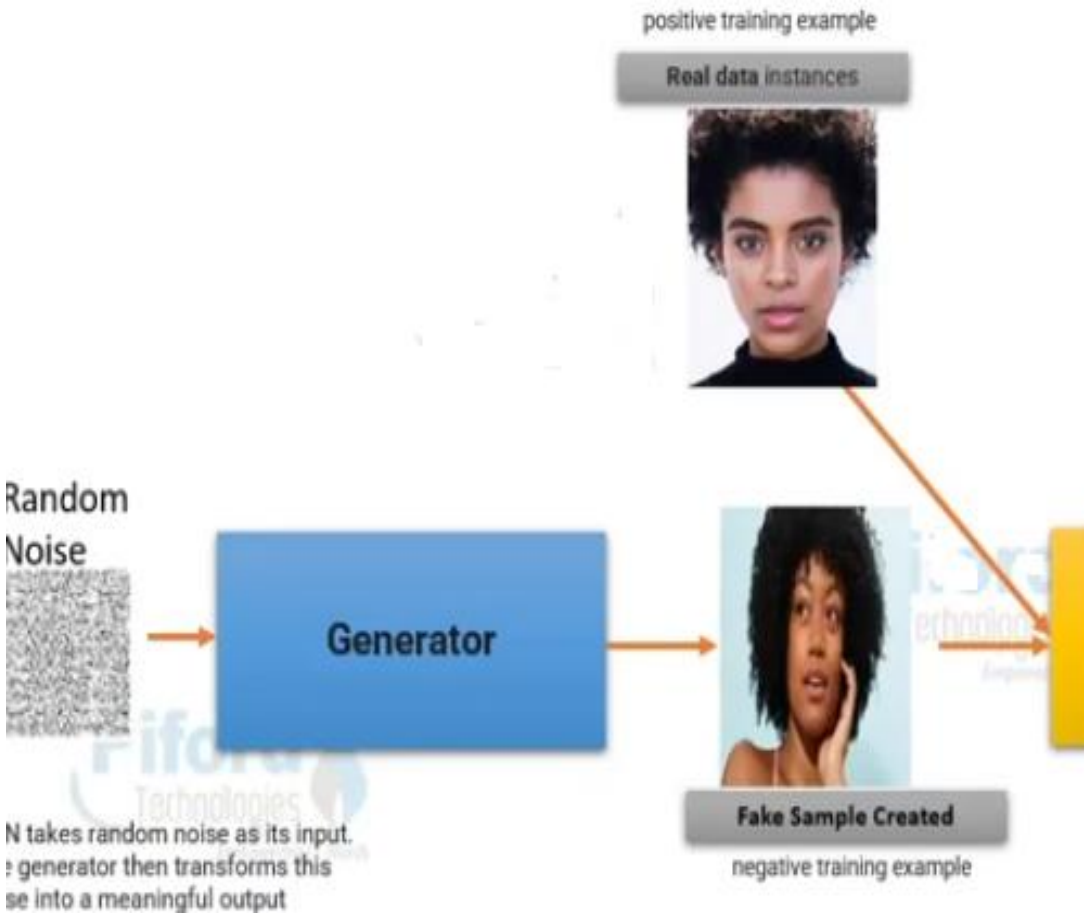
Different phases of GAN training



Initial Phase



After some 20 epochs



After many epochs

Applications of GAN

GANs are very popular in the gaming industry and they can automate the task of creating of 3 D characters and backgrounds to give a realistic feeling

1 **Generating Cartoon Characters**





Using DCGANs, you can create faces of anime and Pokemon characters

2

Generating Human Faces



GANs can be trained on the images of humans to generate realistic faces

3

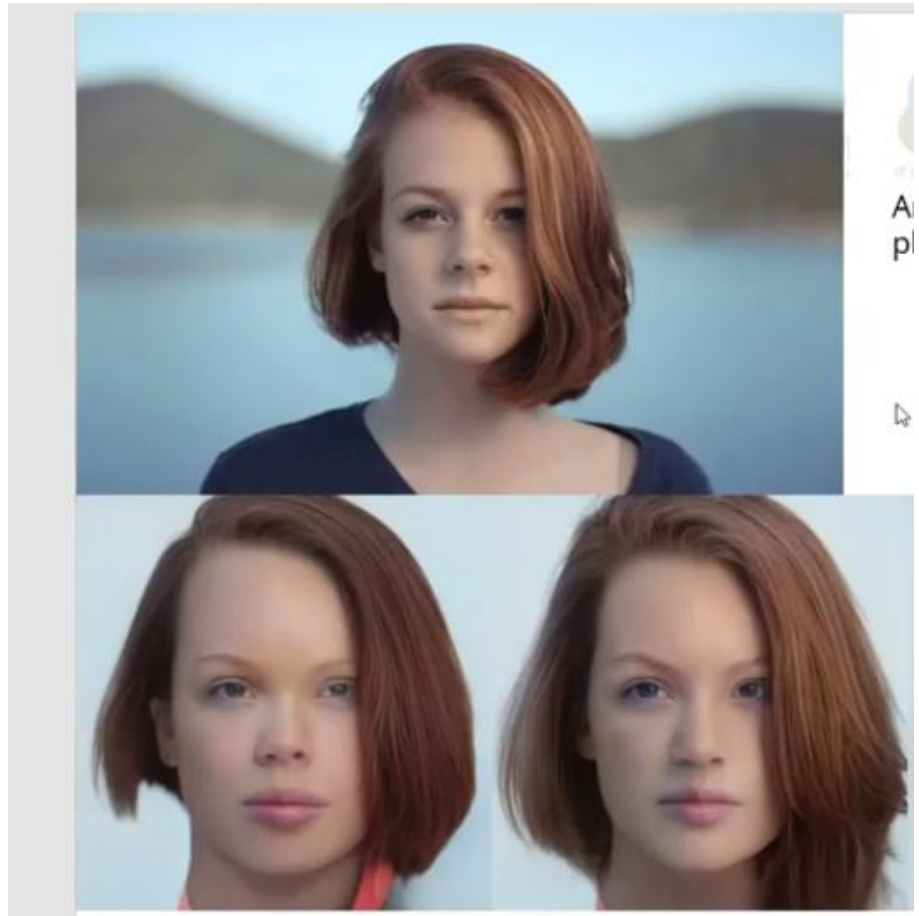
Text to image translation

Bird with a black head, yellow body and a short beak



GANs can build realistic images from textual descriptions of simple objects like birds

- 4) Enhance resolution of images
- 5) Conversion of Photos to Emoji’s
- 6) Face APP – Aging and De-aging can be implemented
- 7)Photograph editing and Reconstruction





Example of Using a GAN to Remove Rain From Photographs Taken from Image De-raining Using a Conditional Generative Adversarial Network

UpSampling

- The Upsampling layer is **a simple layer with no weights that will double the dimensions of input** and can be used in a generative model when followed by a traditional convolutional layer.
- The sole purpose of up sampling layers is **to reduce computations in each layer**, while keeping the dimension of input/output as before.

Challenges Faced By GANs

- Problem of stability between generator and discriminator
- Problem to determine positioning of the objects
- Problem in understanding the perspective
- Problem in understanding global objects

GAN Challenges

- In recent times, GANs has achieved outstanding performance in producing natural images. However, there exist major challenges in training of GANs, i.e.,
- **mode collapse,**
- **non-convergence and**
- **Instability**
- due to inappropriate design of network architecture, use of objective function and selection of optimization algorithm.

- **Convergence**
- As the generator improves with training, the discriminator performance gets worse because the discriminator can't easily tell the difference between real and fake. If the generator succeeds perfectly, then the discriminator has a 50% accuracy.
- This progression poses a problem for convergence of the GAN as a whole: the discriminator feedback gets less meaningful over time. If the GAN continues training past the point when the discriminator is giving completely random feedback, then the generator starts to train on junk feedback, and its own quality may collapse.
- For a GAN, convergence is often a fleeting, rather than stable, state.

Feature Correlation

Uncorrelated Features



Add beard



Correlated Features



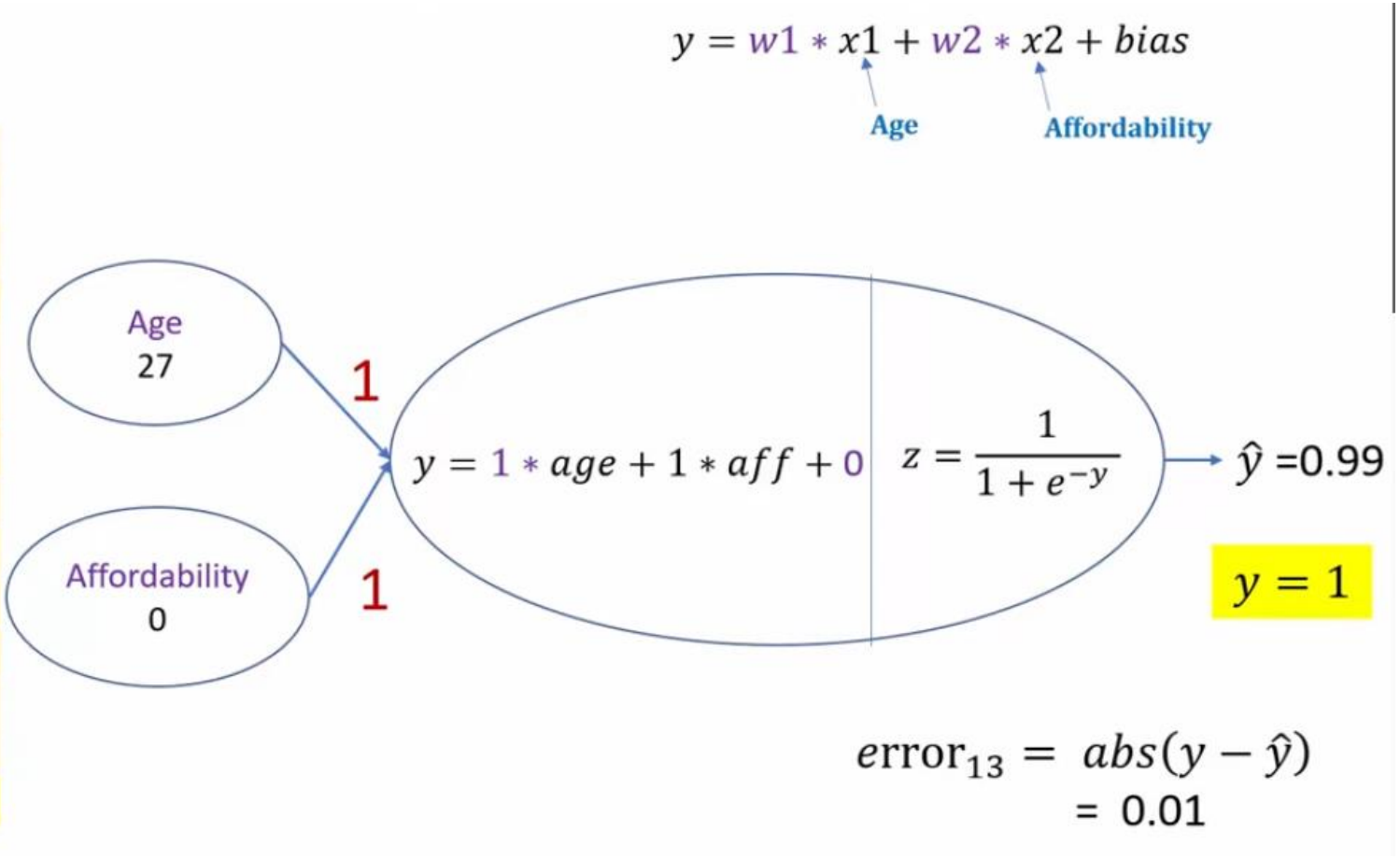
Add beard

Make more masculine



Loss functions

age	affordability	have_insurance
22	1	0
25	0	0
47	1	1
52	0	0
46	1	1
56	1	1
55	0	0
60	0	1
62	1	1
61	1	1
18	1	0
28	1	0
27	0	1



Total error = $error_1 + error_2 + \ldots + error_{13}$

$$= \sum_{i=1}^{n} abs(y_i - \hat{y}_i)$$

$$Mean\ Absolute\ Error\ (MAE) = \frac{1}{n}\sum_{i=1}^{n} abs(y_i - \hat{y}_i)$$

$$\text{Total error} = error_1 + error_2 + \ldots + error_{13}$$

$$= \sum_{i=1}^{n} abs(y_i - \hat{y}_i)$$

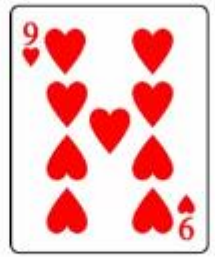
Loss

$$Mean\ Absolute\ Error\ (MAE) = \frac{1}{n}\sum_{i=1}^{n} abs(y_i - \hat{y}_i)$$

Cost Function

Actual Cards





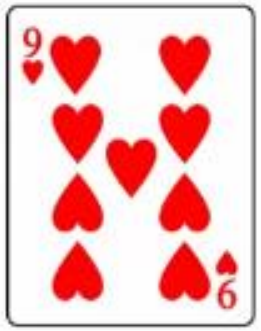


Actual Cards

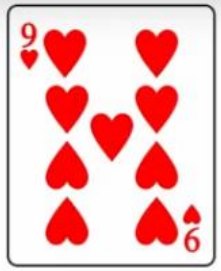
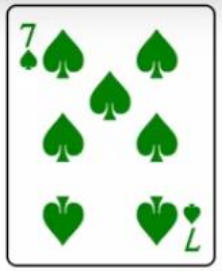


Random Guess



Actual Cards			
Random Guess			
Absolute Error	5	3	1

Mean Absolute Error = 9 / 3 = 3

Actual Cards			
Random Guess			
Squared Error	25	9	1

Mean Squared Error = 35 / 3

Activate Windows
Go to Settings to activate Windows.

$$Mean\ Absolute\ Error\ (MAE) = \frac{1}{n}\sum_{i=1}^{n} abs(y_i - \hat{y}_i)$$

```
model.compile(optimizer='adam',
              loss='mean_absolute_error',
              metrics=['accuracy'])
```

$$Mean\ Squared\ Error\ (MSE) = \frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

```
model.compile(optimizer='adam',
              loss='mean_squared_error',
              metrics=['accuracy'])
```

$$Log\ loss\ or\ binary\ cross\ entropy = -\frac{1}{n}\sum_{i=0}^{n} y_i \log(\hat{y}_i) + (1 - y_i) \quad \cdot \log(1 - \hat{y}_i)$$

Cross entropy

- Cross-entropy is commonly used in machine learning as a loss function.
- Cross-entropy calculate the total entropy between the distributions.
- Cross-entropy is a measure of the difference between two probability distributions for a given random variable or set of events.
- **Entropy** is the number of bits required to transmit a randomly selected event from a probability distribution. A skewed distribution has a low entropy, whereas a distribution where events have equal probability has a larger entropy.

- Neural Network uses optimising strategies like stochastic gradient descent to minimize the error in the algorithm. Error is computed by using a Loss Function. It is used to quantify how good or bad the model is performing.
- These are divided into two categories i.e.Regression loss and Classification Loss.

- **Regression Loss Function**
- Regression Loss is used when we are predicting continuous values like the price of a house or sales of a company.

1.Mean Squared Error

- Mean Squared Error is the mean of squared differences between the actual and predicted value. If the difference is large the model will penalize it as we are computing the squared difference.

- 2.Mean Squared Logarithmic Error Loss

- Suppose we want to reduce the difference between the actual and predicted variable we can take the natural logarithm of the predicted variable then take the mean squared error. This will overcome the problem possessed by the Mean Square Error Method. The model will now penalize less in comparison to the earlier method.

3.Mean Absolute Error Loss

- Sometimes there may be some data points which far away from rest of the points i.e outliers, in case of cases Mean Absolute Error Loss will be appropriate to use as it calculates the average of the absolute difference between the actual and predicted values.

- Binary Classification Loss Function

- Suppose we are dealing with a Yes/No situation like “a person has diabetes or not”, in this kind of scenario Binary Classification Loss Function is used.
- 1.Binary Cross Entropy Loss

- It gives the probability value between 0 and 1 for a classification task. Cross-Entropy calculates the average difference between the predicted and actual probabilities.

Multi-Class Classification Loss Function

- For a dataset like Iris, three-class labels needs to be predicted

  Setosa, Versicolor and Virginia
- In cases where the target variable has more than two classes Multi-Class Classification Loss function is used.
- 1.Categorical Cross Entropy Loss

- These are similar to binary classification cross-entropy, used for multi-class classification problems.

- 2. Kullback Leibler Divergence Loss

- Kullback Leibler Divergence Loss calculates how much a given distribution is away from the true distribution. These are used to carry out complex operations like autoencoder where there is a need to learn the dense feature representation.

Loss Functions for GAN

- GANs try to replicate a probability distribution. They should therefore use loss functions that reflect the distance between the distribution of the data generated by the GAN and the distribution of the real data.
- Two common GAN loss functions are
- **minimax loss**
- **Wasserstein loss**

- A GAN can have two loss functions:
- one for generator training and
- one for discriminator training.
- The generator and discriminator losses are derived from a single measure of distance between probability distributions.
- In both of these schemes, however, the generator can only affect one term in the distance measure: the term that reflects the distribution of the fake data. So during generator training we drop the other term, which reflects the distribution of the real data.

Minimax Loss

$$E_x[log(D(x))] + E_z[log(1 - D(G(z)))]$$

- In this function:

- D(x) is the discriminator's estimate of the probability that real data instance x is real.
- Ex is the expected value over all real data instances.
- G(z) is the generator's output when given noise z.
- D(G(z)) is the discriminator's estimate of the probability that a fake instance is real.
- Ez is the expected value over all random inputs to the generator (in effect, the expected value over all generated fake instances G(z)).
- The formula derives from the cross-entropy between the real and generated distributions.
- The generator can't directly affect the log(D(x)) term in the function, so, for the generator, minimizing the loss is equivalent to minimizing log(1 - D(G(z))).

- Modified Minimax Loss

- Minimax loss function can cause the GAN to get stuck in the early stages of GAN training when the discriminator's job is very easy. The generator loss can be modified so that the generator tries to maximize log D(G(z)).

- **Wasserstein Loss**
- This loss function depends on a modification of the GAN scheme (called "Wasserstein GAN" or "WGAN") in which the discriminator does not actually classify instances.
- For each instance it outputs a number. This number does not have to be less than one or greater than 0, so we can't use 0.5 as a threshold to decide whether an instance is real or fake. Discriminator training just tries to make the output bigger for real instances than for fake instances.
- Because it can't really discriminate between real and fake, the WGAN discriminator is actually called a "critic" instead of a "discriminator".

- Critic Loss: D(x) - D(G(z))
- The discriminator tries to maximize this function. In other words, it tries to maximize the difference between its output on real instances and its output on fake instances.
- Generator Loss: D(G(z))
- The generator tries to maximize this function. In other words, It tries to maximize the discriminator's output for its fake instances.
- In these functions:
- D(x) is the critic's output for a real instance.
- G(z) is the generator's output when given noise z.
- D(G(z)) is the critic's output for a fake instance.
- The output of critic D does not have to be between 1 and 0.

- **Benefits**
- Wasserstein GANs are less vulnerable to getting stuck than minimax-based GANs, and avoid problems with vanishing gradients.